

Resource Access Protocol and its Application in Autonomous Traffic Control System”

Abdullah Al Forkan¹

Contents

1	Motivation	3
1.1	Introduction	3
2	Foundation	4
2.1	Resource Access Protocol	4
2.2	Resource Access Conflicts	4
2.2.1	Priority Inversion	5
2.2.2	Deadlock	5
2.2.3	Timely Anomalies	5
2.3	Types of Resource Access Protocols	5
2.3.1	Non-Preemptive Protocol	5
2.3.2	Highest Locker Priority Protocol	6
2.3.3	Priority Inheritance Protocol	6
2.3.4	Priority Ceiling Protocol	7
3	Application in Autonomous Traffic Control system	8
3.1	Scenario	8
3.2	Problem	8

¹ Abdullah-al.forkan@stud.hshl.de

3.3	Solution	9
3.4	UPPAAL Implementation	10
4	Challenges	10
5	Conclusion	11
6	Declaration of Originality	11

Abstract: Resource Access Protocols are responsible for managing the system resource. They set rules for a system about how the tasks or processes should access the resources. This paper explores the common types of resource access protocols like non-preemptive, highest locker priority, priority inheritance, and priority ceiling and popular conflicts like priority inversion, deadlock, and timely anomalies. Then a practical implementation of an autonomous traffic control system is performed to illustrate the resource sharing behaviours. A type of resource access protocol is used to solve the problems, which is the priority inheritance protocol. Following the protocol, a UPPAAL model is simulated. Then all the challenges have been addressed. Finally, this paper ends with a conclusion. The goal is to create a basic understanding of the resource-sharing rules on different types of resource access protocols and finally apply that knowledge to real-life scenarios.

1 Motivation

1.1 Introduction

We are living in a world now, where data is everything and all processes of the system share the same data to maintain connectivity. Generally, a resource can be a data structure, a set of variables, a main memory area, a file, or a set of registers of a peripheral device. Embedded systems use these resources to process any task. Single-processor systems were introduced first, where only one processor access the resources to execute tasks. However, when multiprocessor systems were invented, resource sharing became complex. Multiple processors work faster, the problem was how to manage them. Different tasks in the processors need different resources, but it would eventually happen that multiple tasks request the same resource. To solve and manage all these problems resource access protocols were introduced.

Resource access protocols are laws that tell a system how it should behave while accessing the resources. Various types of protocol are included here and systems can implement any one of them according to their task requirements. There are two parts to a task execution process. One is the non-critical section, where it does not require any resources and another one is the critical section, where it asks for resources. Each protocol manages them in its own way, some execute each process independently without preempting and some preempt. Failing to implement the proper protocol system could destroy the whole purpose of the system. An example of resource access protocol failure would be the Mars Pathfinder robot failure, which happened in 1997 [Pa10]. The robot was resetting again and again after landing, delaying the actual purpose of the system. The problem occurred due to a priority inversion problem while processing the tasks. The mission was NASA's first discovery mission and incurred 265 million dollars. The expected lifetime of the robot for the mission was very short, so it can be understood that this kind of problem can result in a big loss.

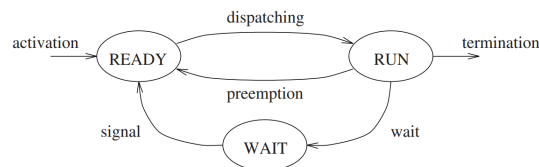


Fig. 1: States of a process

The basic structure of a process has three states ready, run, and wait as can be seen in fig:1 [BU97]. Initially, it starts in the ready state and then goes to the run state. If it is blocked before completion, it goes to the wait state. When it leaves the waiting state, it does not go to the run state directly, instead goes to the ready state again to be reassigned by the CPU. The CPU calculates the priority of the processes according to the protocol laws and executes them.

In this study resource access protocol is discussed in a very simple way and then it is implemented in an autonomous traffic control system scenario. Resource sharing is a crucial part in autonomous systems and failing to implement proper resource management can lead to dangers.

2 Foundation

The foundation is built by discussing the resource access protocol, resource sharing conflicts, and some commonly used types of resource access protocols. [BU97]

2.1 Resource Access Protocol

Resources are the most important parts of a system that are requested by processes to execute programs. Resources can be data structures, variables, main memory areas, files, sets of integers, input/output units, etc. When these resources are shared with multiple tasks are called shared resources. A process executed under mutual exclusion constraints is called a critical section and any task request to enter a critical section needs to wait until the resources are free. This waiting period is referred to as blocked on those resources. A

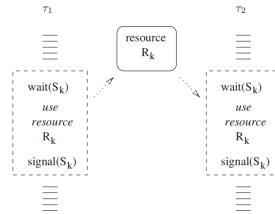


Fig. 2: Resource shared between two tasks

shared resource system can be understood in fig:2, where the key elements are Resource R_k , Semaphore S_k , and Tasks T_1 and T_2 . Resource R_k is the shared resource that the tasks T_1 and T_2 want to access. Semaphore S_k is a synchronization system that controls the access of resource R_k . It can be accessed through two kernel primitives wait and signal. Each task wants to access resource R_k and has to start with wait S_k and finish with signal S_k . [BU97]

2.2 Resource Access Conflicts

Resource access protocol handles the task in a priority-based system, where higher priority tasks can access the resources first, and then comes the lower priority tasks. However, it has some conflicts that can ruin the whole purpose of the system. [Li09]

2.2.1 Priority Inversion

Priority inversion is a common problem in resource access protocol systems, where low-priority jobs behave like high-priority jobs and vice versa. This situation occurs when a low-priority job locks the resources and prevents other jobs from accessing the resources. Higher-priority jobs might preempt the low-priority jobs that are holding the resources, but they can not enter the critical section because of this priority inversion problem and can be blocked for an infinitely long time.

2.2.2 Deadlock

When two or more tasks are waiting for a resource that is held by another task, it results in a deadlock. For instance, this deadlock situation occurs if task-1 holds a resource 'x' and waits for resource 'y', while task-2 holds resource 'y' and requests resource 'x'. All the tasks get blocked indefinitely, shutting down the system.

2.2.3 Timely Anomalies

Priority inversion and deadlock make the tasks miss deadlines which causes timing anomalies. When a task has a dedicated deadline and a priority inversion problem does not allow it to access the resources, it misses the deadline and results in timely anomalies. Similarly, a deadlock in the system also delays the deadline and causes timely anomalies.

2.3 Types of Resource Access Protocols

There are many resource access protocols, each designed for specific purposes in concurrent systems. Some common types are discussed below [BU97]:

2.3.1 Non-Preemptive Protocol

The non-preemptive protocol is a great solution for an indefinite priority inversion problem that prevents preemption during the execution of any critical section. When task T_i enters the critical section, it raises the priority of task T_i to the maximum priority and completes the critical section without any interruption. The formula is:

$$P_i(R_k) = \max_h \{P_h\}$$

This protocol works very efficiently for tasks with short critical sections. However, the problem arises when the critical section is long, which creates unnecessary blocking. In

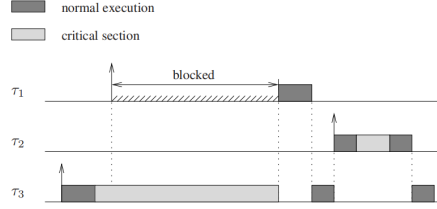


Fig. 3: Non-Preemptive Protocol

fig:3, the scheduling of NPP can understood, T_3 stops after the complete execution of the critical section. T_3 has a long critical section that blocks T_1 for a long period.

2.3.2 Highest Locker Priority Protocol

The highest locker priority protocol (HLP) is an improved version of NPP(Non-Preemptive Protocol). It only raises the priority of the tasks, which are using the same resource R_K . The formula is:

$$p_i(R_k) = \max_h \{P_h | T_h \text{ uses } R_k\}$$

It is the same as NPP, except the part ' T_h uses R_k '. It means that the priority between users of resource R_k . In fig:4, it can be observed that T_3 enters the critical section and T_1

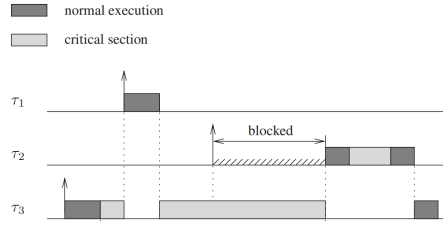


Fig. 4: Highest Locker Priority Protocol

preempts T_3 since it does not have any critical section and has higher priority. But T_3 blocks T_2 , because it has higher priority and they share the same resource.

2.3.3 Priority Inheritance Protocol

PIP (Priority inheritance protocol) occurs When a lower-priority job blocks a higher-priority job, the lower-priority job is promoted to the priority level of the higher-priority job. Tasks

with the same priority are processed in a First Come First First Serve manner. It follows the following formula:

$$p_j(R_k) = \max\{P_j, \max_h \{P_h | T_h \text{ is blocked on } R_k\}\}$$

When a task T_i is blocked by T_j that holds the semaphore, T_j inherits the priority of T_i . According to the fig:5, T_1 is the highest priority task, then comes T_2 , and T_3 respectively.

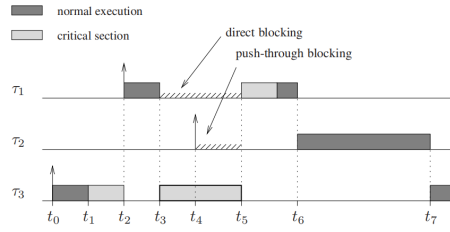


Fig. 5: Priority Inheritance Protocol

Lower-priority task T_3 enters the critical section and blocks all other tasks for that critical section. At time t_2 , task T_1 preempted T_3 to execute its non-critical section maintaining the original priority. But when it comes to the critical section, T_3 still has the first priority. At time t_4 , task T_2 requests for a non-critical execution, which is denied by T_3 . As soon as T_3 finishes its execution, T_1 goes back to the original first priority and executes its critical section.

2.3.4 Priority Ceiling Protocol

PCP (Priority ceiling protocol) is the same as PIP (Priority Inheritance Protocol), where the job is scheduled in a priority-based manner. It is an upgrade to the priority inversion problem for multiple semaphores and avoids deadlocks, and chained blocking. It upgrades PIP, allowing a lock request condition on a free semaphore. This lock request condition does not allow a task to enter a critical section if there are locked semaphores with higher priority. The formula is:

$$p_j(R_k) = \max\{P_j, \max_h \{P_h | T_h \text{ is blocked on } R_k\}\}$$

This formula is exactly the same as PIP, where task T_j inherits T_i . The difference is the following:

$$C(S_k) \stackrel{\text{def}}{=} \max_i \{P_i | S_k \in \sigma_i\}.$$

Besides the task priority, PCP introduces the semaphore priority rule. Each semaphore S_k is assigned a priority ceiling $C(S_k)$, which is equal to the highest priority of the tasks that can lock it. In Fig:6, T_3 starts and lock semaphore S_C . Then T_2 comes with higher priority at time t_1 and requests for semaphore S_C at time t_2 . But S_C is already locked by T_3 which

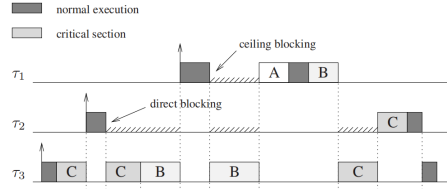


Fig. 6: Priority Ceiling Protocol

makes T_3 higher priority than T_2 . After blocking T_2 , T_3 continues executing S_C . T_3 starts a nested semaphore S_B as it was free and gets preempted by task T_1 with the highest priority at time t_4 . T_1 finishes its normal execution and requests for S_A at time t_5 . S_A is not locked by any task. But because of the ceiling block, T_1 can not start this semaphore. According to the protocol, S_B is not unlocked yet and has a higher priority than S_A . So, T_3 inherits T_1 and resumes its execution. T_3 finishes execution at time t_6 and T_1 becomes the first priority again. After T_1 completes its execution, T_3 starts computing S_C as it still blocks T_2 . Finally, T_2 can access S_C at time t_8 and continues executing.

3 Application in Autonomous Traffic Control system

Resource Access Protocols find applications in a variety of systems where shared resources need to be managed efficiently. Some key implementations can be seen in Real-Time Systems, Embedded Systems, Operating Systems, Distributed Systems, Database Systems, Network Protocols, Autonomous Systems, etc. In this paper, an autonomous traffic control scenario is examined [QZdJ22].

3.1 Scenario

In an autonomous traffic control system on a four-way intersection, vehicles communicate with the server and create a priority queue to cross the road. Each vehicle sends a message of the desired direction to the server and the server allocates them in a priority-based manner as shown in fig:7. Assume a scenario, where multiple vehicles are waiting to cross in each direction and the goal is to implement a protocol that achieves maximum parallelism for those vehicles which means the server allows a maximum number of vehicles to pass at the same time efficiently. For example, all vehicles waiting to go east and west direction can proceed together, blocking others.

3.2 Problem

In a regular priority-based queue, several problems can occur like priority inversion, deadlock, and timely anomalies, which are discussed in resource access conflicts. As

Priority 1	Car 1
Priority 2	Car 2
Priority 3	Car 3
Priority 4	Car 4

Fig. 7: Priority Queue

each vehicles are executed separately, that makes the server resources locked for each direction. This creates a time inversion problem where a low-priority car could be executing indefinitely, blocking the other vehicles. There is a big possibility of deadlock, where two or more vehicles are waiting for a resource that is held by another vehicle. Finally, these problems kill the efficiency of transportation and create timely anomalies. similar problems can also be seen in the Mars pathfinder problem, where priority inversion occurred. This resulted in a system reset repeatedly, damaging the whole system.

3.3 Solution

Priority inheritance protocol would be a great solution for priority inversion, deadlock, and efficiency problems. The solution can be understood from fig:8, assuming J_1 , J_2 , and

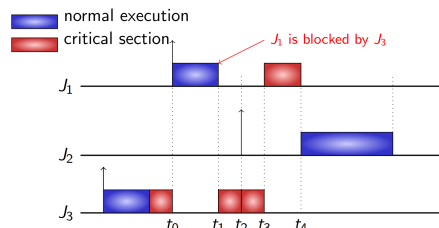


Fig. 8: Priority Inheritance Protocol Solution

J_3 as separate maximum parallelism scenarios (Jobs), where multiple vehicles cross the intersection in each job. Here, J_1 has the first priority, J_2 second, and J_3 third. Blue means approaching and red means crossing.

- J_3 approaches the junction first and locks the critical section (crossing area).
- Then, J_1 approaches at time t_0 and requests to cross at t_1 . But it cannot go to the critical section as J_3 has already locked it. So, J_3 blocks J_1 , inheriting the first priority.
- J_2 requests to approach at time t_2 , which is denied by J_3 by pushed through blocking.

- As soon as J_3 is crossed at t_3 , it goes back to the regular priority and J_1 gets back its first priority again.
- Finally J_1 completes the critical section at t_4 and J_2 starts approaching.

So, the priority inheritance protocol solves the problem avoiding priority inversion, deadlock, and timely anomalies. It makes the whole autonomous traffic control system very efficient and danger-free.

3.4 UPPAAL Implementation

Priority inheritance protocol solution of the autonomous traffic control system is implemented through UPPAAL simulation to analyze the behaviors of the system. The simulation is given in fig:9

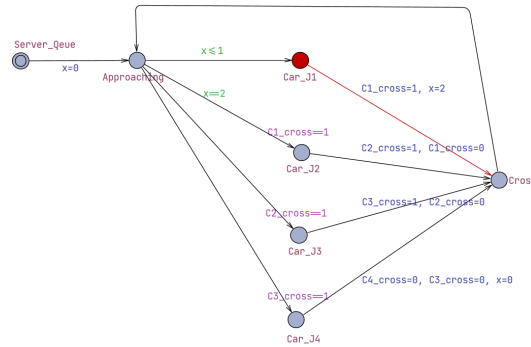


Fig. 9: UPPAAL Simulation

4 Challenges

Resource-sharing protocol types were quite complex to understand. But it became very easy to understand when compared with real-life scenarios. All the types have a lot of similarities in functionalities which was also a problem. It was tricky to differentiate them. Practical implementation is a great way to understand any research, but it was challenging to find a proper implementation that follows the protocol. Autonomous system is a huge area and finding the right implementation was hard. Finally, it was possible to find the right scenario, problem, solution, and simulation.

5 Conclusion

Every system has resources and processors are dependent on them. After this study, it is clear that resource sharing can be quite complex and proper protocol has to be applied to ensure smooth processing of the system. Various resource sharing problems occurred in past, most of them happened due to priority inversion, deadlock, or timely anomalies. Priority inversion makes the system behave oddly and breaks the priority law. Deadlock totally stops the system and timely anomalies happen when a task misses its dedicated deadline. These failures not only waste millions of dollars, and huge amount of time, but also can endanger human life. Proper investigation and testing can help identifying these problems beforehand and then the solution can be implemented on time.

Some common types of resource access protocols have been discussed like non-preemptive protocol, highest locker priority protocol, priority inheritance protocol, and priority ceiling protocol. Non-preemptive protocol is mostly used where tasks have to be executed without any interruption, timely response to the events is mandatory, and interruptions leads to instability. Highest locker priority protocol is not used much in everyday electronic, it is mostly used in real-time operating systems, ensuring reliability, predictability, and safety. Priority inheritance protocol provides great solutions for priority inversion in real-time operating systems. It is widely used in FreeRTOS, Satellite systems, medical devices, automotive systems, etc. Priority ceiling protocol is used where priority inversions and deadlocks cause serious issues. It is mostly used in FreeRTOS-MPU (memory protection unit), industrial control systems, automotive system, and where priority inheritance needs an upgrade.

In the implementation an autonomous traffic control scenario has been optimised using priority inheritance protocol. This fixed the system from the issues like priority inversion, deadlock, and timely anomalies. This application is really crucial here, because these resource access conflicts can cause accidents on the roads. This also increased the overall efficiency of the transportation system through maximum parallelism, locking the resources for each parallelism scenario.

Finally, it can be said that behind every great system, there is resource access protocol that maintains the laws and orders of the system. More research on this area would have a great impact on modern electronics.

6 Declaration of Originality

I, Abdullah Al Forkan, hereby declare that this paper is my original work and that I have not submitted it for any other degree or qualification. I have acknowledged all sources of information and have cited these sources in accordance with the Latex referencing system. I have not copied or plagiarized any material, nor have I knowingly allowed others to copy or plagiarize my work. I understand that plagiarism is a serious offense and that disciplinary

action will be taken against any student found guilty of plagiarism.

Bibliography

- [BU97] BUTTAZZO, Giorgio C.: Resource access protocols. In (BUTTAZZO, Giorgio C., ed.): Hard real-time computing systems, volume 416 of Real-time systems, pp. 181–223. Kluwer Academic Publishers, Boston and Dordrecht and London, 1997.
- [Li09] Liu, Jane W. S.: Real-time systems. Prentice Hall and Pearson Education [distributor], Upper Saddle River, N.J. and London, 2009.
- [Pa10] Pathan, Risat: Report_MarsPathFinder. Report_MarsPathFinder:Attachments/Report_MarsPathFinder.pdf:application/pdf, 2010.
- [QZdJ22] Qiao, Jianglin; Zhang, Dongmo; de Jonge, Dave: Priority-Based Traffic Management Protocols for Autonomous Vehicles on Road Networks. In (Long, Guodong; Yu, Xinghuo; Wang, Sen, eds): AI 2021: Advances in Artificial Intelligence, volume 13151 of Lecture Notes in Computer Science, pp. 240–253. Springer International Publishing, Cham, 2022.